

Git and GitHub for Research Labs

A practical command-line and MATLAB tutorial

Ehsan Eqlimi

Columbia University New York, USA

[eqlimi.ehsan@gmail.com] (<mailto:eqlimi.ehsan@gmail.com>)

1 Goal

By the end of this tutorial, you should be able to:

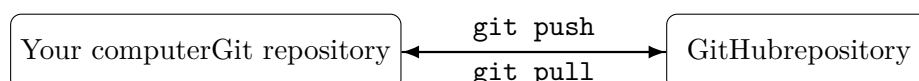
- create and clone Git repositories;
- connect Git to GitHub using SSH;
- track MATLAB, Python, R, and other research code;
- create meaningful commits;
- synchronize work between computers;
- collaborate using branches;
- inspect and recover previous versions;
- use Git from MATLAB when appropriate.

2 Git and GitHub

Git is a version-control system that runs locally on your computer.

GitHub is a service that hosts Git repositories and provides collaboration features.

A useful model is:



Git records project history locally. GitHub provides a remote copy and collaboration platform.

3 Install and Configure Git

Check whether Git is installed:

```
git --version
```

Configure your commit identity:

```
git config --global user.name "Your Name"
git config --global user.email "[your@email.com](mailto:your@email.com)"
```

Check the configuration:

```
git config --global --list
```

The name and email identify your Git commits. They are separate from your GitHub authentication credentials.

4 Configure SSH for GitHub

SSH allows Git to authenticate with GitHub using an SSH key.

If you do not already have a key:

```
ssh-keygen -t ed25519 -C "[your@email.com](mailto:your@email.com)"
```

Display the public key:

```
cat ~/.ssh/id_ed25519.pub
```

Add the public key to:

GitHub → Settings → SSH and GPG keys

Never share:

```
~/.ssh/id_ed25519
```

The private key must remain secret.

Test the connection:

```
ssh -T [git@github.com](mailto:git@github.com)
```

Note

SSH avoids GitHub username/password authentication for normal Git operations, but an SSH key may still require its passphrase unless an SSH agent or keychain is handling it.

5 Create and Clone a Repository

Create a repository on GitHub, for example:

```
EEG_Beta_Project
```

Clone it using its SSH URL:

```
git clone [git@github.com](mailto:git@github.com):YOUR_USERNAME/EEG_Beta_Project.git
cd EEG_Beta_Project
```

Check the remote:

```
git remote -v
```

You should see an SSH URL beginning with:

```
[git@github.com](mailto:git@github.com):
```

If a repository was originally cloned using HTTPS, change it to SSH:

```
git remote set-url origin [git@github.com](mailto:git@github.com):YOUR_USERNAME/
EEG_Beta_Project.git
```

6 The Basic Git Workflow

The essential workflow is:

```
git pull

# Edit and test your files

git status
git diff

git add .
git commit -m "Describe the change"
git push
```

The commands have different roles:

Command	Purpose
<code>git pull</code>	Retrieve and integrate remote changes
<code>git status</code>	Show repository state
<code>git diff</code>	Show uncommitted changes
<code>git add</code>	Stage changes for the next commit
<code>git commit</code>	Record a version locally
<code>git push</code>	Send local commits to GitHub

A good commit describes a meaningful change:

```
git commit -m "Fix frontal beta power calculation"
```

Avoid vague messages such as:

```
git commit -m "stuff"
```

7 History and Recovery

View the history:

```
git log --oneline
```

Inspect a commit:

```
git show COMMIT_ID
```

Compare commits:

```
git diff COMMIT_ID_1 COMMIT_ID_2
```

Git therefore provides a structured history instead of files such as `analysis_final.m`, `analysis_final2.m`, and `analysis_really_final.m`.

8 Branches

Use a branch when developing a substantial change:

```
git switch -c new-analysis
```

After finishing the work:

```
git switch main
git merge new-analysis
```

Branches allow experimental work to remain separate from the main branch until it is ready.

9 Research Projects and .gitignore

A repository might contain:

```
EEG_Beta_Project/  
|  
|-- code/  
|-- scripts/  
|-- docs/  
|-- figures/  
|-- README.md  
|-- .gitignore
```

Do not automatically put all research data into GitHub.
Potentially inappropriate repository contents include:

- participant-identifying information;
- confidential or restricted data;
- passwords and API keys;
- very large generated files;
- temporary files.

A project-specific `.gitignore` might contain:

```
# MATLAB temporary files  
  
*.asv  
  
# Operating-system files  
  
.DS_Store  
Thumbs.db  
  
# Project-specific data  
  
data/  
raw_data/
```

If your project requires it, specific data formats such as `*.mat` can also be ignored:

```
*.mat
```

However, do this only if all such files should be excluded.

Warning

Adding a file to `.gitignore` does not stop Git from tracking a file that was already committed. Such a file must first be removed from Git tracking, for example with:

```
git rm --cached filename
```

Always follow your institution's rules for research data storage.

10 Working on Multiple Computers

Each computer should have its own clone of the repository.

The basic sequence is:

```
Computer A:
git pull
work
git add .
git commit -m "Describe the change"
git push

Computer B:
git pull
work
git add .
git commit -m "Describe the change"
git push
```

The important rule is:

Pull before starting work on another computer.

If two computers modify the same lines, Git may report a merge conflict. Resolve the conflict carefully and check the scientific meaning of the result before committing.

11 MATLAB Daily Workflow

MATLAB can execute normal Git commands directly:

```
!git status
!git pull
!git add .
!git commit -m "Describe the change"
!git push
```

Recent MATLAB releases also provide programmatic Git functionality, including `gitclone`, `gitrepo`, `status`, `add`, `commit`, `push`, and `log`. These functions were introduced in R2023b.

For example:

```
repo = gitclone( ...
" [git@github.com] (mailto:git@github.com):YOUR_USERNAME/EEG_Beta_Project.git");

status(repo)
```

An existing repository can be represented with:

```
repo = gitrepo("path/to/EEG_Beta_Project");
```

MATLAB also provides Git integration through the GUI, including **Home** → **New** → **Git Clone**.

12 MATLAB Start-of-Day Script

A simple script can pull the latest version before work:

```
clear;
clc;

% Set this to the repository on the current computer.
projectFolder = "/path/to/EEG_Beta_Project";

cd(projectFolder);

status = system("git status");
if status ~= 0
error("This folder is not a valid Git repository.");
end

status = system("git pull");
if status ~= 0
error("git pull failed. Resolve the problem before working.");
end

disp("Repository is up to date.");
```

For multiple computers, set `projectFolder` appropriately for each machine rather than relying on hard-coded usernames and computer names.

13 MATLAB End-of-Day Script

A simple end-of-day script is:

```
clear;
clc;

projectFolder = "/path/to/EEG_Beta_Project";
cd(projectFolder);

status = system("git status");
if status ~= 0
error("This folder is not a valid Git repository.");
end

system("git diff --stat");

system("git add .");

disp("Files staged for commit:");
system("git status");

commitStatus = system( ...
'git commit -m "Update research code"');

if commitStatus ~= 0
warning("No commit was created. Check git status.");
end

pushStatus = system("git push");

if pushStatus ~= 0
error("git push failed. Check the repository and SSH connection.");
end
```

```
disp("Changes pushed successfully.");
```

For important scientific changes, replace the generic commit message with something meaningful, for example:

```
git commit -m "Fix frontal beta power calculation"
```

Warning

Do not make `git push -force` part of an automatic daily workflow. Force-pushing can rewrite remote branch history.

14 Do Not Delete .git

The hidden directory

```
.git
```

contains the local repository metadata and history.

Do not delete it during normal work.

Do not repeatedly run:

```
rm -rf .git
git init
git remote remove origin
git remote add origin ...
```

These commands are for special repository-reconfiguration or recovery situations, not normal daily work.

15 Version Control Is Not a Complete Backup Strategy

Git and GitHub are useful for versioning and sharing source code, but they should not be treated as the complete backup strategy for a research project.

For example, if research data are excluded by `.gitignore`, they are not stored in the GitHub repository.

Use an appropriate institutional or research-data storage and backup system for large or sensitive datasets.

A useful separation is:

Git/GitHub

Research datasets and other large files

Source code, scripts, documentation, configuration Data storage/h

16 Useful Commands

Command	Purpose
	<code>git status</code>
Show repository state <code>git diff</code>	Show uncommitted changes <code>git add file</code>
Stage a specific file <code>git add .</code>	Stage eligible changes <code>git commit -m "message"</code>
Create a local commit <code>git pull</code>	Retrieve and integrate remote changes <code>git push</code>
Send commits to the remote repository <code>git log -oneline</code>	Show compact history <code>git remote -v</code>
Show remote URLs <code>git switch -c name</code>	Create and switch to a branch <code>git merge name</code>
Merge a branch	

17 Minimal Cheat Sheet

Start

```
git pull
```

Work

Edit and test your code.

Finish

```
git status
git diff
git add .
git commit -m "Describe the change"
git push
```

pull → work → add → commit → push

Take-Home Message

Git provides version control for research code.

GitHub provides a remote repository and collaboration platform.

SSH provides key-based authentication.

The most important habits are:

1. Pull before starting work.
2. Commit meaningful changes.
3. Push completed work.
4. Keep the `.git` directory.
5. Use a project-specific `.gitignore`.
6. Never commit secrets or inappropriate research data.

7. Maintain a separate backup strategy for research data.